Pelican Tours — Real AWS, Real Budget

One small business, one real cloud, one $100 budget that has to survive the whole semester. You'll deploy the same app three different ways — as raw infrastructure, as managed platform, and as serverless — and learn why each exists.

Individual project | 4 milestones · Classes 4 · 7 · 9 · 11 | AWS Academy Learner Lab | No credit card — $100 class credit | Live demo on Class 12

The client

Pelican Tours runs kayak, snorkel, and airboat trips out of Tampa, Florida — 9 guides, 6 boats, and the best manatee encounter rate on the Gulf Coast. Bookings live in a spiral notebook by the register. The "website" is a Facebook page.

Then a travel influencer's manatee video hit 40 million views with the caption *"book with Pelican Tours."* The phone melted. Owner **Marissa Roydon** hired a web kid who vanished mid-project, leaving one thing behind: a working little booking app (your `starter-app/`).

Marissa's brief to you: *"Put it on real cloud infrastructure. It has to survive the next viral video, our photo archive is drowning my laptop, my guides need proper logins, and I can't spend real money until bookings do. Oh — and hurricane season starts in June."*

Your Learner Lab budget **is** Marissa's budget: $100 for the semester. If you exhaust it, the account shuts off — so every architectural choice you make is also a financial one.

One app, three architectures

The spine of the project: you deploy the *same booking app* three ways across the semester, and by the end you can explain from experience when each service model makes sense.

	Architecture	Service model	You manage	Built in
1	Everything on one EC2 instance (then an Auto Scaling group behind a load balancer)	IaaS — rehost	OS, runtime, scaling, health	M1
2	Frontend on S3 static hosting · API on EC2/ALB · photos in S3 with lifecycle tiers	Managed services — replatform	Less OS, more configuration	M2–M3
3	API Gateway + Lambda + DynamoDB · frontend stays on S3	Serverless — refactor	Code and configuration only	M4

The starter app is pre-cut for this journey: the API/static seam is marked in `server.js`, the frontend takes a configurable API base for S3 hosting, and bookings are held in memory *on purpose* — you'll watch that state vanish during scaling and finally fix it properly with DynamoDB in M4.

Milestones & rubrics

M1 · Rehost: Marissa Goes Live

DUE CLASS 4 | 25 pts | Obj 1 | Obj 2 | Obj 3 | Obj 4

The booking app moves from the vanished web kid's laptop to real infrastructure — then survives its first fake viral video.

- [ ] **Lab orientation:** complete the Learner Lab setup guide; journal your first budget reading and the answer to: who is actually paying for the hardware under your $100, and what NIST trait is that?
- [ ] **Launch the instance (IaaS):** EC2 t3.micro, Amazon Linux 2023, `LabInstanceProfile`, `user-data.sh` from the starter app. Security group: HTTP 80 from anywhere, SSH 22 from your IP only. Journal: what did you have to decide that a SaaS user never sees? Screenshot the working site + `/api/health`.
- [ ] **Sizing memo:** defend t3.micro with numbers (vCPU, RAM, burst credits, on-demand price/hr from the pricing page) and name what you'd move to — and what it would cost — if Marissa's traffic 10×'d.
- [ ] **The viral-video drill (elasticity):** create a launch template + Auto Scaling group (min 1 / desired 2 / max 3) behind an Application Load Balancer. Refresh `/api/health` through the ALB DNS and screenshot `served_by` rotating across instances. Then terminate one instance *manually* and watch the ASG heal itself — screenshot the timeline. Scale desired back to 1 (budget!).
- [ ] **The lost-state experiment:** place a booking, then scale/replace instances and show the booking is gone. Journal: where did it go, why is in-memory state incompatible with elasticity, and which AWS services fix it? (You'll implement the fix in M4.)
- [ ] **Broad network access:** book a tour from your phone (cellular, not Wi-Fi) via the ALB URL. One photo, one sentence.
- [ ] **Build vs buy (SaaS lens):** half-page appraisal — should Marissa just use a booking SaaS (FareHarbor, Peek, etc.)? Compare on cost model, control, lock-in, and multi-tenancy; state when you'd advise each path. PaaS lens: what did the ALB + ASG manage *for* you, and what would a full PaaS have taken off your plate?
- [ ] **Cost ledger opens:** table of every resource you're running, its hourly price, and your measured budget burn for the week. Predict M2's burn; you'll grade your own prediction next week.

```
# M1 target architecture
#
#   phone/browser --> ALB (port 80) --> Auto Scaling group (1-3 x t3.micro)
#                                       each runs starter-app via user-data
#
# Order of operations (console, us-east-1):
# 1. EC2 -> Launch templates -> create from your working single instance
#    (AMI: Amazon Linux 2023 · t3.micro · LabInstanceProfile · user-data)
# 2. EC2 -> Target groups -> HTTP:80, health check path /api/health
# 3. EC2 -> Load balancers -> ALB, internet-facing, 2+ AZs, your target group
# 4. EC2 -> Auto Scaling groups -> min 1 / desired 2 / max 3, attach target group
# 5. Visit http://ALB-DNS-NAME/api/health repeatedly -> served_by rotates
# 6. Terminate an instance -> watch ASG replace it (screenshot before/after)
# 7. Desired capacity -> 1, then End Lab. Budget line in journal.
```

```
# Where user-data downloads your app from (pick ONE, do it before launching):
#
# Option A — public GitHub repo (simplest):
#   1. Push the starter-app folder to a public repo you create
#   2. Your zip URL is:
#      https://github.com/YOUR-USERNAME/YOUR-REPO/archive/refs/heads/main.zip
#   3. Paste that into APP_ZIP_URL in user-data.sh
#
# Option B — S3 bucket in your lab account:
#   1. Zip the folder:  zip -r starter-app.zip starter-app
#   2. S3 -> Create bucket (any unique name) -> Upload starter-app.zip
#   3. Select the object -> Presigned URL (12 hours) -> paste into APP_ZIP_URL
#      (an instance relaunching after the URL expires will fail — Option A avoids that)
```

M1 rubric

Criterion	Pts
Instance + user-data deploy works; security group and sizing reasoned like an engineer	6
ALB/ASG demo: rotation shown, self-healing shown, scaled back down	8
Lost-state experiment run and correctly explained	4
Build-vs-buy appraisal uses SaaS/PaaS/IaaS distinctions correctly	4
Journal + cost ledger complete, budget lines present	3

M2 · Identity & Data: The Photo Archive and the Guides DUE CLASS 7 25 pts Obj 5 Obj 6

Marissa's 180 GB of manatee photos get a real home with real tiers, the frontend moves to S3 static hosting, and you appraise identity solutions from inside a federated sandbox.

- [ ] **You ARE a federation user:** diagram your own login path (Canvas/Academy → Vocareum → temporary AWS credentials → console) and label the identity provider, the service provider, and the trust. Compare to corporate SSO into AWS.
- [ ] **Read the role you live in:** inspect `LabRole` in IAM — pick three allowed statements and one explicit deny, translate each to plain English, and explain how least privilege shows up in your own sandbox (what were you stopped from doing, and why is that good design?).
- [ ] **Author a least-privilege policy:** author (JSON, in your repo) a least-privilege policy for a hypothetical "photo-uploader" guide role: put/get on `manatee-photos/*` only, no delete, no other buckets. Explain each statement. (Learner Lab may block attaching it — authoring and defending it is the assessed part; attach only if your lab allows.)
- [ ] **IDaaS appraisal:** Marissa's 9 guides need logins for a future admin panel. Compare Amazon Cognito vs Okta/Entra ID vs "passwords in the app" on: MFA options, provisioning/deprovisioning (a guide quits mid-season!), cost at 9 users vs 900, and ops burden. Recommendation memo, half page. *Stretch (+2): if your Learner Lab allows Cognito, stand up a user pool with MFA and enroll one guide.*
- [ ] **Object storage with tiers:** S3 bucket for the photo archive; upload sample "photos"; add a lifecycle rule (Standard → Standard-IA at 30d → Glacier Flexible at 90d); enable versioning and demonstrate restoring an overwritten file; share one photo via a time-limited presigned URL. Cost the full 180 GB archive per tier per month (pricing page) — show Marissa the before/after.
- [ ] **Block vs object vs file, on real services:** table mapping EBS / S3 / EFS to what each holds in your architecture (root volume · photos+frontend · shared guide files), with one workload each is wrong for. Take an EBS snapshot now (you'll need it in M3). *EFS: create, mount on your instance, write one shared file — then delete the mount target and file system the same session (it bills hourly).*
- [ ] **Frontend to S3 (replatform begins):** host `public/` as an S3 static website with a `config.js` pointing `API_BASE` at your ALB. The site now survives every EC2 instance dying. Journal: which halves of the app now sit in which service model?
- [ ] **Data classification register:** 6+ data types (bookings, guest names, photos, configs, credentials, logs…) with sensitivity, storage service, tier, and who can access. Note: why must the lab hold only fictional guest data?

```
# M2 — frontend on S3 static hosting, step by step
# 1. S3 -> Create bucket (unique name) -> UNCHECK "Block all public access"
# 2. Upload everything inside starter-app/public/
# 3. Create config.js in the bucket with one line:
#      window.API_BASE = "http://YOUR-ALB-DNS-NAME";
# 4. Bucket -> Properties -> Static website hosting -> Enable
#      index document: index.html   -> note the website endpoint URL
# 5. Bucket -> Permissions -> Bucket policy (public read of objects):
{
  "Version": "2012-10-17",
  "Statement": [{
    "Sid": "PublicRead",
    "Effect": "Allow",
    "Principal": "*",
    "Action": "s3:GetObject",
    "Resource": "arn:aws:s3:::YOUR-BUCKET-NAME/*"
  }]
}
# 6. Open the website endpoint — tours should load from your ALB (the API
#    already sends CORS headers). Booking flow should work end to end.
```

M2 rubric

Criterion	Pts
Federation diagram + LabRole reading show real understanding of identity plumbing	5
Least-privilege policy JSON correct and defended	4
IDaaS appraisal compares real options on real criteria	4
S3 lifecycle + versioning + presigned URL demonstrated; archive costed per tier	6
EBS/S3/EFS differentiation on real resources; snapshot taken; EFS cleaned up	3
S3 static frontend live via config.js; classification register sound	3

M3 · Hurricane Season: Images, Hardening & the Drill DUE CLASS 9 25 pts Obj 7

Virtualization gets real — golden images, instance-type economics, a hardening pass — and then a simulated hurricane takes out us-east-1.

- [ ] **Instance-type safari (virtualization economics):** using the EC2 pricing/instance pages, build a table for t3.micro, c5.large, r5.large, and one instance 10× t3.micro's price: vCPU, RAM, ratio, price/hr, and the workload each shape exists for. Then: which does the booking app actually need, and what does overprovisioning cost per year? Connect to how the hypervisor lets AWS sell one physical box as many shapes.
- [ ] **Golden image:** bake your configured instance into a custom AMI. Launch a fresh instance from it with *no user data* and time boot-to-serving. Compare against the user-data path; journal when imaging beats bootstrapping (and what "AMI drift" would mean six months from now).
- [ ] **Containers appraisal:** you just built a VM image; compare with containerizing the app (startup time, density, image size, isolation, orchestration needs). Recommend VM vs container vs serverless for each Pelican Tours component — you'll test your serverless prediction in M4.
- [ ] **Hardening pass:** (1) security-group audit — tightest rules that still work, screenshot before/after; (2) S3 Block Public Access everywhere except the website bucket, and say why the website bucket is the exception; (3) verify encryption at rest on EBS + S3 (it's on by default — so what, exactly, does it protect against, and what not?); (4) fix the root/port-80 flag in `user-data.sh` (the comment tells you where); (5) confirm IMDSv2-only on your instances.
- [ ] **Shared-responsibility matrix, AWS edition:** for EC2 vs S3 vs (M4's) Lambda: who patches the hypervisor, the OS, the runtime, the app code, and who owns data + access config in each. One sharp paragraph: which of *your* past checkboxes were "security OF the cloud" vs "security IN the cloud"?
- [ ] **Targets first:** RTO + RPO for the booking site, the photo archive, and guide files — justified in Marissa's business terms (a lost booking vs a lost decade of photos are different tragedies), and the DR tier each implies (backup-restore / pilot light / warm standby / multi-site).
- [ ] **Hurricane drill:** photos bucket gets cross-region replication to us-west-2 (verify an object lands there). Then simulate losing the region: terminate *all* instances, delete the ASG. Recover in **us-west-2**: copy your AMI across, launch, repoint `config.js`. Time every step. Report actual RTO/RPO vs your targets in a 1-page incident report — graded on the honesty of the analysis, not on hitting your numbers. Clean up us-west-2 completely the same session, and journal what a *warm standby* there would have cost per month vs your restore time.
- [ ] **SLA reality check:** EC2, S3, and the ALB each publish an SLA — find the numbers, compute the allowed monthly downtime (Session 9 method), and compare with the availability your single-region M1 design could honestly promise Marissa. What does the gap between "AWS's SLA" and "your architecture's availability" teach?

M3 rubric

Criterion	Pts
Instance-type economics + golden-image timing show virtualization understood from the buyer's chair	6
Containers appraisal reasoned per component	3
Hardening pass complete with before/after evidence; shared-responsibility matrix sharp	7
Hurricane drill executed cross-region; measured RTO/RPO vs targets; honest incident report; full cleanup	7
SLA math correct and the architecture-vs-SLA gap articulated	2

M4 · Serverless Refactor & the Reckoning

DUE CLASS 11 · DEMO CLASS 12 | 25 pts | Obj 8

The API becomes functions, bookings persist in DynamoDB, monitoring alerts reach your inbox — and three architectures face one cost comparison.

- [ ] **Refactor (SOA for real):** re-implement the API as Lambda functions behind API Gateway (`LabRole` as execution role) with bookings in a DynamoDB table — the in-memory flaw from M1, finally fixed. Frontend on S3 just changes `config.js`. Prove bookings persist by placing one and reading it back after a fresh deployment.
- [ ] **API design sheet:** document 5+ REST endpoints (method · path · purpose · sample request/response · status codes). Design and implement one *new* endpoint (e.g., `GET /api/availability?date=`). Journal: what makes this loosely coupled — name the contract, and what each side can now change without asking the other.
- [ ] **Loose coupling, live:** break the API deliberately (throttle the stage to 0 or break a route) — the S3 frontend shows its degraded banner instead of dying. Screenshot, restore, recover. Bonus insight: which pieces of your M3 DR plan just became AWS's problem?
- [ ] **Monitoring + alerting:** CloudWatch alarm on API Gateway 5xx (or Lambda errors) → SNS email; trigger it for real and screenshot the email. Define 5 metrics + thresholds you'd watch if Pelican Tours had 50 tour companies as tenants; note the on-call reality for a company of one.
- [ ] **The reckoning (cost engineering):** one spreadsheet, three architectures (M1 EC2+ASG · M2 hybrid · M4 serverless) priced at three traffic levels: 100, 10k, and 1M bookings/month (pricing calculator + your actual measured burn as the sanity check). Recommendation memo to Marissa: which architecture at which stage of her growth, and the trigger points for switching.
- [ ] **Everything-as-evidence:** final architecture diagram (all three generations, before/after), repo complete, cost ledger closed out with total semester burn vs $100.
- [ ] **Live demo (Class 12):** present and demo live from your lab account. Chaos moment: the instructor picks one thing to break (an instance, the API stage, or a photo); you show the blast radius, the alert, and the recovery.

```
# M4 — serverless refactor, one endpoint worked end to end (repeat the pattern)
#
# 1. DynamoDB -> Create table:  name Bookings · partition key bookingId (Number)
# 2. Lambda -> Create function: name pelican-bookings · Runtime Node.js 20
#    -> Change default execution role -> Use existing role -> LabRole
# 3. Paste this as index.js (handles GET and POST /api/bookings):

const { DynamoDBClient } = require('@aws-sdk/client-dynamodb');
const { DynamoDBDocumentClient, PutCommand, ScanCommand } = require('@aws-sdk/lib-dynamodb');
const ddb = DynamoDBDocumentClient.from(new DynamoDBClient({}));
const resp = (statusCode, obj) => ({
  statusCode,
  headers: { 'Content-Type': 'application/json', 'Access-Control-Allow-Origin': '*' },
  body: JSON.stringify(obj)
});
exports.handler = async (event) => {
  if (event.requestContext.http.method === 'POST') {
    const b = JSON.parse(event.body || '{}');
    if (!b.tourId || !b.name) return resp(400, { error: 'tourId and name are required' });
    const booking = { bookingId: Date.now(), tourId: b.tourId, name: b.name,
                      guests: b.guests || 1, bookedAt: new Date().toISOString() };
    await ddb.send(new PutCommand({ TableName: 'Bookings', Item: booking }));
    return resp(201, booking);
  }
  const out = await ddb.send(new ScanCommand({ TableName: 'Bookings' }));
  return resp(200, out.Items);
};

# 4. API Gateway -> Create HTTP API -> Add integration: your Lambda
#    Routes:  GET /api/bookings  and  POST /api/bookings  -> same integration
#    CORS: allow origin *, methods GET,POST,OPTIONS, header Content-Type
#    Deploy (default stage) -> copy the Invoke URL
# 5. Second function for the catalog: pelican-tours returns tours.json the
#    same way (no DynamoDB needed) on GET /api/tours — same resp() pattern
# 6. Update config.js in your S3 bucket:
#      window.API_BASE = "https://XXXX.execute-api.us-east-1.amazonaws.com";
# 7. Alarm: SNS -> Create topic + email subscription (confirm the email!) ->
#    CloudWatch -> Alarms -> Create: metric = your Lambda Errors >= 1 -> notify topic.
#    Break the route on purpose, watch the email arrive.
```

Stretch (up to +3 bonus): Infrastructure as Code — a CloudFormation template that stands up your M1 stack (launch template, ASG, ALB) in one deploy. Demo: delete the stack, redeploy, working site, zero console clicks. Explain why this changes DR math entirely.

M4 rubric

Criterion	Pts
Serverless refactor works end-to-end; DynamoDB fixes the state flaw and you can say why	8
REST design sheet follows conventions; new endpoint sensible; coupling explained via the contract	5
Break/recover demo + alarm fired to a real inbox	4
Three-architecture cost model honest, sourced, sanity-checked against measured burn; memo decisive	5
Live demo: clear story, working system, chaos moment survived, questions handled	3

Class-by-class checkpoints

Class · Day	Due at the start of class (journal evidence, budget line included)	Feeds
3 · Thu	Project introduced · accept the Learner Lab invite · start the setup guide this weekend — *nothing due, but M1 lands Monday*	—
4 · Mon	**M1 due** — lab access verified + first budget reading · repo + journal · EC2 via user data · ALB/ASG viral-video drill · sizing memo · lost-state note · cost ledger opened	M1
5 · Tue	**Due (written):** federation diagram of your own lab login · LabRole reading	M2
6 · Thu	**Due:** photo-uploader policy JSON · S3 archive with lifecycle + versioning · EBS snapshot taken	M2
7 · Mon	**M2 due** — presigned share · EBS/EFS exercise · frontend live from S3 · classification register	M2
8 · Tue	**Due (written):** RTO/RPO targets per system · shared-responsibility matrix draft	M3
9 · Thu	**M3 due** — instance-type table · golden AMI timed · hardening pass · hurricane drill with incident report · us-west-2 cleaned up	M3
10 · Mon	**Due:** Lambda + API Gateway routes working · DynamoDB wired in · CloudWatch alarm fired to your inbox	M4
11 · Tue	**M4 due** — API design sheet, break/recover evidence, three-architecture cost model + recommendation memo; the heavy build was Monday's checkpoint	M4
12 · Thu	**Final Day — live demos** (presentation only)	M4